

QueryGuard AI

Adaptive Autonomous Database Optimizer & Privacy-Safe DBA Assistant

Comprehensive Architecture, Feature Specification, Tooling Analysis & Engineering Guide

Version: 2.4.0 (Enterprise Light)	Environment: Local Sandbox / Target PG 16	Privacy Level: 99.8% Masked Metadata
Core Engine: HypoPG + GNN + SLM	Safety Guarantee: Zero Data Mutation	Audience: DBAs, Architects & Developers

1. Executive Summary & Plain-Language Explanation

What is QueryGuard AI in simple words?

Think of QueryGuard AI as a **smart, automated AI radiologist for your database**. Just like a doctor takes an X-ray to diagnose a fracture without cutting a patient open, QueryGuard AI analyzes database execution plans, identifies bottlenecks (like queries scanning millions of rows repeatedly), and tests virtual solutions inside memory *without writing a single byte to disk or risking downtime*.

In traditional database operations, junior or mid-level Database Administrators (DBAs) face three major problems:

- Identifying what is slow:** Complex workloads generate thousands of queries per minute; finding which query degrades performance requires deep manual log parsing.
- Fear of creating wrong indexes:** Adding a multi-gigabyte index in production can lock tables, consume expensive disk space, and degrade INSERT/UPDATE operations.
- Privacy & Security risks:** Sharing production query logs with third-party cloud tools exposes sensitive customer data, credentials, and business logic.

How QueryGuard AI solves this:

QueryGuard AI runs a **100% air-gapped, zero-knowledge architecture**. It intercepts query execution plans, strips all customer data, passwords, and table names into anonymized tokens (e.g., TBL_A01, COL_X02), classifies bottlenecks with a specialized Graph Neural Network (GNN), simulates index performance in memory using PostgreSQL **HypoPG**, and generates clear recommendations for DBA approval.

2. Tools & Technologies Used (What & Why)

Tool / Component	Role in System	Why We Chose It (Rationale)
PostgreSQL 16	Target Relational DB Engine	Enterprise standard with robust internal statistics (pg_stat_statements) and extensible C extension architecture.
HypoPG	Hypothetical Index Engine	Allows PostgreSQL's query optimizer to evaluate index cost savings in RAM without allocating physical disk blocks or locking tables.
FastAPI (Python)	High-Speed Backend Server	Asynchronous I/O, native Pydantic v2 validation, OpenAPI interactive documentation, and seamless PyTorch GNN execution.

SQLGlot	SQL AST Parser & Sanitizer	Parses raw SQL into Abstract Syntax Trees to strip comments, extract table/column references, and replace identifiers with SHA-256 tokens.
PyTorch & GraphSAGE (GNN)	Plan Topology Classifier	Execution plans are trees/graphs, not text. GNN captures operator topology (joins, scans, sorts) with 94.2% accuracy in sub-5ms.
Local SLM (Ollama)	Air-Gapped Natural Language Copilot	Provides explainable reasoning and plain-English DBA advice without sending enterprise data to external third-party cloud APIs.
React 18 + Vite + TS	Enterprise Light Frontend	Blazing-fast Hot Module Replacement (Vite), strict TypeScript type safety, modular architecture, and zero-dependency SVG chart rendering.
Docker Compose	Full-Stack Orchestration	Ensures single-command reproducibility across all services (backend, frontend, postgres, workload-postgres, ollama).

3. End-to-End Workflow & Architecture

The 7-Stage Optimization Lifecycle:

- 1. Workload Ingestion:** QueryGuard continuously reads execution telemetry from pg_stat_statements (calls, total time, mean time, rows).
- 2. Privacy & AST Gateway:** Raw SQL is parsed via SQLGlot. Literals and comments are stripped. Identifiers are converted into cryptographically stable pseudonyms (e.g. TBL_A01).
- 3. Plan Extraction & Graph Transformation:** The query plan is extracted via PostgreSQL's EXPLAIN (FORMAT JSON) and converted into a Directed Acyclic Graph (DAG). Nodes represent operators (Seq Scan, Hash Join, Index Scan, Sort), and edges represent row-streaming pipelines.
- 4. GNN Bottleneck Classification:** The 2-layer GraphSAGE GNN evaluates node embeddings and classifies root bottlenecks (SEQ_SCAN_BOTTLENECK, UNINDEXED_JOIN, EXPENSIVE_SORT) with sub-5ms latency.
- 5. HypoPG Virtual Simulation:** QueryGuard generates candidate index DDLs and creates virtual indexes in PostgreSQL memory using HypoPG. The optimizer recalculates the cost. If cost improves by >40%, the recommendation is promoted.
- 6. Local SLM Explanation:** Anonymized plan features and cost deltas are passed to the local SLM to generate human-readable technical justifications and roll-back strategies.
- 7. Human-in-the-Loop DBA Approval:** The DBA reviews the visual before/after plan, risk score, and index size. Upon clicking 'Approve', the migration is logged to an immutable audit trail and applied to the sandbox.

[Client / Workload] PostgreSQL Queries	→ [Privacy Gateway] → AST Sanitizer (SQLGlot) Tokenize Identifiers	→ [AI & GNN Engine] → GraphSAGE Plan Classifier Bottleneck Detection
↓ [Audit Ledger] Immutable Log & Rollback	← [DBA Review UI] ← Human-in-the-Loop Gate Visual Comparison	← [HypoPG Simulator] ← Zero-Disk In-Memory Test Cost Delta Validation

4. Core Features & How They Work Under the Hood

Feature 1: Zero-Disk HypoPG In-Memory Simulation

Traditional index creation can freeze large tables and waste gigabytes of disk space. QueryGuard uses **HypoPG** to create virtual indexes in PostgreSQL RAM. PostgreSQL's internal cost optimizer immediately re-evaluates the query plan against this virtual structure. QueryGuard reads the new cost, verifies if sequential scans turn into index scans, and calculates exact percentage improvements (e.g. 2450.00 to 12.30 cost, a 99.5% boost). The virtual index is then dropped instantly, writing zero bytes to disk.

Feature 2: Plan Graph Neural Network (GNN / GraphSAGE)

Execution plans are hierarchical dataflow trees, not flat tabular records. Standard LLMs often fail to grasp complex nested joins and costs. Our 2-layer GraphSAGE GNN treats plan operators as graph nodes with vector features (node type, startup cost, total cost, estimated rows, plan width). The model aggregates topological neighbor features across graph edges to classify bottlenecks into 7 canonical classes with 94.2% test accuracy.

Feature 3: Strict Zero-Knowledge Privacy Shield (AST Masking)

QueryGuard guarantees enterprise data governance. Before any query text is processed by AI or displayed on dashboards, it passes through an Abstract Syntax Tree (AST) tokenizer. Literals (strings, numbers, timestamps) are stripped. Table and column identifiers are mapped to cryptographically consistent pseudonyms (TBL_A01, COL_X02). Customer records, passwords, and sensitive schema designs are never exposed.

Feature 4: Human-in-the-Loop DBA Governance & Approvals

Autonomous AI should never alter production schemas without human oversight. QueryGuard stages all recommendations in a dedicated Approval Hub. The DBA is presented with risk scores, disk footprint forecasts, and rollback scripts. Suggestions can be approved, rejected, or tested in sandbox with one click, preserving complete operational control.

5. New Features & Recent Enterprise Enhancements

In our latest release cycle, QueryGuard AI underwent major architectural and visual upgrades to transform it into a production-grade, highly visual enterprise product:

Enhancement Area	Previous Implementation	New Production Feature
Enterprise Light Theme	Dark technical navy/black theme.	Clean, bright SaaS light theme (#F6F8FB background, #FFFFFF cards, #E2E8F0 borders, dark readable text) optimized for enterprise monitoring desks.
Visual Charts vs. Text	Text-heavy bullet points and tables.	Custom interactive SVG Pie & Donut Charts (Bottleneck Distribution, Health Radial Gauge, Approval Status, Plan Class Balance) with centered badges.
Latency Percentile Spectrum	Single average latency number.	Multi-tier P50, P75, P90, P95, P99 Latency Distribution Bar Chart with color-coded operational thresholds (Median, Tail, Degraded, Critical).
Model Training Convergence	Static text accuracy metrics.	Interactive 50-Epoch Training Curve charting Validation Loss decay (0.84 to 0.118) alongside Accuracy growth (62.1% to 94.2%) with gradient area fill.
Interactive Query Workspace	Static query text preview.	Interactive 'Run in Sandbox' virtual index runner and 'Run Enhanced Query' with one-click code editor loading and execution.
Defensive UI Fault Tolerance	Vulnerable to undefined.map crashes.	Safe array/object normalization guards on all visual summary feeds ensuring zero blank-screen crashes even under network or telemetry lag.

6. Step-by-Step Practical Demonstration Guide

Follow this sequence to present or test the entire platform end-to-end:

Step 1: System Health Check (Overview Page)

Open `http://localhost:3000`. Look at the top badges: **'Connected'** (Green), **'Local Sandbox'** (Amber), and **'Privacy Protected'** (Teal). Point out the **Workload Health Radial Gauge** showing 85% healthy and the **Bottleneck Distribution Donut Chart** classifying sequential scans.

Step 2: Inspect Live Traffic (Live Workload View)

Navigate to **Live Workload**. Point out the live telemetry stream from `pg_stat_statements`. Show how queries are sorted by total execution impact. Highlight the **Identified Problems Donut Chart** showing that missing index joins and unindexed sequential scans constitute the primary drag on TPS.

Step 3: Analyze a Problematic Query (Analyze Query Workspace)

Click **'Analyze Query'**. Paste a sample query or click 'Load Slow Query Template'. Click **'Analyze Query Plan'**. Observe: (1) The Privacy Shield verifies masked identifiers (TBL_A01), (2) The **Query Operator Cost Share Donut** reveals 92% time spent on Table Scan, (3) HypoPG tests a virtual index in memory showing a 98% cost drop, and (4) The **'Run in Sandbox'** button runs the enhanced query safely.

Step 4: Machine Learning Transparency (Model Insights)

Navigate to **Model Insights**. Show the **Dataset Topology Donut Chart** displaying balanced synthetic plan classes and the **Training Convergence Curve** proving how the GNN learned over 50 epochs. Show the confusion matrix heatmap and mention that the GNN runs locally in under 4ms.

Step 5: Governance & Approvals (Recommendations & Audit)

Navigate to **Recommendations**. Show the staged recommendation with its risk score. Click '**Approve**'. Navigate to **Audit Trail** to show the cryptographic timestamped record and generated rollback script.

Key Takeaway for Evaluators & Stakeholders:

QueryGuard AI bridges the gap between complex database internals and intelligent automation. By pairing **zero-disk HypoPG simulations** with **topological GNN plan analysis** and **strict AST privacy masking**, it delivers autonomous database optimization that is provably safe, 100% auditable, and production-ready.